《资治通鉴 · 周纪》智能比对 Agent 关键核心技术讲义稿

面向大一新生的从零讲解版

2026 年 3 月 26 日

目录

1 这份讲义讲什么 **4**

2 先把问题讲明白 **4**

2.1 这个项目要解决什么问题 4

2.2 为什么不能只用关键词搜索 5

3 系统总架构 **5**

4 核心技术一：文本数据准备 **6**

4.1 为什么数据准备是第一技术 6

4.2 项目如何下载和清洗古籍 6

4.3 为什么这一步属于“核心技术” 7

5 核心技术二：古文友好的文本预处理 **7**

5.1 按语剥离 7

5.2 繁简转换 7

5.3 句子切分 7

6 核心技术三：分块 Chunking **8**

6.1 为什么要分块 8

6.2 项目里的分块策略 8

6.3 为什么要 overlap 8

6.4 一个可以记住的经验 8

7 核心技术四：向量化与本地向量数据库 **9**

7.1 什么是向量化 9

7.2 Embedding 模型在项目中怎么用 9

7.3 向量数据库为什么选 ChromaDB 9

7.4 索引建立的流程 10

8 核心技术五：混合检索与重排 10
8.1 为什么不是只做向量检索 10
8.2 项目做了哪三层检索 10
8.3 第一层：特征提取 11
8.4 第二层：关键词加权打分 11
8.5 第三层：语义召回 11
8.6 第四层：局部片段抽取 12
8.7 第五层：窗口评分与惩罚机制 12
8.8 这一步教会我们什么 12

9 核心技术六：Diff 差异比对 13
9.1 为什么要做 Diff 13
9.2 技术原理是什么 13
9.3 为什么输出成 HTML 13

10 核心技术七：本地 LLM 解释层 13
10.1 为什么检索之后还要 LLM 13
10.2 项目如何调用大模型 14
10.3 Prompt 设计有什么特点 14
10.4 为什么强调“本地” 14

11 核心技术八：Web 前端与交互封装 15
11.1 为什么还需要 Gradio 15
11.2 这不是“花哨界面”，而是产品化 15
11.3 一个值得注意的工程细节 15

12 核心技术九：鲁棒性与工程质量 15
12.1 懒加载与资源控制 15
12.2 持久化索引 15
12.3 错误恢复 15
12.4 测试 16

13 从头到尾走一遍完整样例 16
13.1 样例输入 16
13.2 系统内部会发生什么 16
13.3 这一步最想让你看懂什么 17

14 这套技术方案为什么成立 17
14.1 它抓住了任务本质 17
14.2 它没有迷信大模型 17
14.3 它适合教学 17

15 局限与改进方向 18

16 给大一新生的最终总结 18

1 这份讲义讲什么

这份讲义基于项目源码写成，目标不是只告诉你“它用了哪些库”，而是让你真正明白这个项目为什么能工作、每一步在解决什么问题、不同技术之间是怎样接起来的。

这个项目本质上是一个**本地运行的中文古籍 RAG 系统**，但它并不只是“检索再问大模型”。它把任务拆成了四件事：

1. 把史料文本准备干净；
2. 把史料切成适合检索的小块并建立向量索引；
3. 先检索最相关的原始史料，再精细重排；
4. 把“原始史料”和“《资治通鉴》改写文本”做差异比对，并交给本地 LLM 解释改写意图。

如果你只记一句话，可以记成：

输入《资治通鉴》一段文字 → 检索可能对应的史料 → 比较哪里删了、哪里改了、哪里加了 → 让本地大模型给出解释。

2 先把问题讲明白

2.1 这个项目要解决什么问题

很多同学第一次看到“大模型项目”，会以为核心是“问模型一个问题”。其实不是。这个项目的真正目标是：

> 给定《资治通鉴 · 周纪》中的一段文本，自动去原始典籍中找到最可能的来源片段，然后分析司马光是如何改写原文的。

这里至少有三个难点：

1. **史料很多**：原始来源可能来自《左传》《国语》《史记》《战国策》等不同文本。
2. **措辞不完全一样**：司马光会删、改、并、缩，不能只靠字符串完全匹配。
3. **输出不能只是“找到像的文本”**：还要解释“为什么这样改写”。

所以，这不是普通搜索，也不是单纯聊天，而是一个“**检索 + 比对 + 解释**”的复合任务。

2.2 为什么不能只用关键词搜索

如果只靠关键词搜索，会遇到几个问题：

- 古文存在繁简差异，比如“显王/顯王”“颜率/顏率”；
- 同一事件会有不同表述，比如“兴师临周”和“發兵至周”；
- 一段长史料里只有中间几句真正相关，整段拿来会很吵。

所以项目没有只做关键词搜索，而是用了**混合检索**：

- 一部分靠实体词和事件词做“硬匹配”；
- 一部分靠向量相似度做“语义匹配”；
- 最后再对局部片段做精细重排。

3 系统总架构

项目入口主要在两个文件中：

- `zztj_agent.py`：核心逻辑；
- `app.py`：Gradio Web 界面。

系统主流程可以概括为：

```
用户输入《资治通鉴》文本
        |
        v
剥离按语（如“臣光曰”）
        |
        v
检查本地向量索引是否已存在
        |
        +-- 不存在 -> 读取 sources/ -> 清洗 -> 分块 -> 向量化 -> 写入 ChromaDB
        |
        v
对输入文本做特征提取
        |
        v
混合检索：关键词召回 + 向量召回
        |
        v
片段级重排，选出最相关史料
        |
```

```
        v
Diff 对比：显示删减/新增/改写
        |
        v
调用本地 LLM 解释改写意图
        |
        v
Gradio 页面展示报告
```

源码里把这个总流程落实成了一个主函数：

```
analyze_zztj_text(input_text)
```

这也是整个项目最值得读懂的函数，因为它把索引、检索、Diff、LLM 全串起来了。

4 核心技术一：文本数据准备

4.1 为什么数据准备是第一技术

很多初学者觉得“模型最重要”。但这个项目最先决定效果上限的，其实是**数据准备**。因为如果原始史料里有大量导航栏、编辑标记、百科说明、乱码，那么：

- 分块会乱；
- 向量会脏；
- 检索会偏；
- Diff 会失真；
- LLM 的解释也会跟着跑偏。

4.2 项目如何下载和清洗古籍

`download_classics.py` 负责批量下载古籍文本。它做了两件事：

1. 用 `curl` 从 Wikisource 拉取网页；
2. 从 HTML 中提取正文，并去掉页面噪音。

其中，`clean_extracted_text()` 和 `_clean_source_text()` 都体现了一个关键思想：

> 先把“给人看的网页”还原成“给机器处理的纯文本”。

它们会去掉的内容包括：

- 返回目录、姊妹计划、百科条目等导航信息；
- “编辑”标记；
- 某些括注和说明文字；
- 多余空白、换行和页面噪音。

4.3 为什么这一步属于“核心技术”

因为 RAG 不是魔法，它非常依赖输入质量。你可以把它理解成：

$$\text{最终效果} \approx \text{数据质量} \times \text{检索质量} \times \text{生成质量}$$

其中任何一项接近零，整体效果都会很差。

5 核心技术二：古文友好的文本预处理

5.1 按语剥离

《资治通鉴》中有时会出现“臣光曰”“臣光按”这样的编者按语。项目在检索前会先调用：

```
_prepare_retrieval_text(input_text)
```

它的作用是：

- 如果检测到按语，就把正文和按语分开；
- 检索时只用正文，不让按语污染查询；
- 报告里再提示“已自动忽略按语”。

这件事很重要，因为按语往往是司马光的评论，不一定属于原始史料本身。

5.2 繁简转换

项目考虑了“显/顯”“颜/顏”“赵/趙”这类问题。它优先用 `OpenCC` 做繁简转换；如果本机没有这个库，就退化为一个手工映射表。

这背后体现了两个工程思想：

1. **算法上容错**：同一实体的不同字形应该视为同一对象；
2. **工程上降级**：可选依赖缺失时，程序仍尽量可运行。

5.3 句子切分

项目没有按英文那种空格分词，而是按古文常见句读来切。可理解为分隔符集合：

```
。!?; \textbackslash n
```

对应函数是：

```
_split_text_sentences(text)
```

这是因为古文检索里，**句子**通常比“固定字数”更接近语义单位。

6 核心技术三：分块 Chunking

6.1 为什么要分块

向量数据库不是直接存整部《史记》，而是存很多个较短文本块。原因有三：

1. 一整卷文本太长，向量会把很多无关信息揉在一起；
2. 查询时真正相关的往往只是其中一小段；
3. Diff 比对也更适合和短片段进行。

6.2 项目里的分块策略

项目参数写得很明确：

- `CHUNK\_SIZE = 280`
- `CHUNK\_OVERLAP = 70`

分块函数是：

```
chunk_text(text, chunk_size=280, overlap=70)
```

它不是简单“每 280 个字切一刀”，而是：

1. 先清洗文本；
2. 再按段落切；
3. 段内再按句子切；
4. 如果句子太长，再做滑动窗口切分；
5. 块和块之间保留重叠，避免上下文丢失。

6.3 为什么要 overlap

假设一个事件刚好跨在两个块的边界上，如果没有重叠，模型可能在两个块里都只看到“半个事件”，导致检索失败。

所以 overlap 的作用可以理解成：

> 让相邻文本块共享一部分上下文，降低“切断语义”的风险。

6.4 一个可以记住的经验

文本分块没有绝对标准，但常见原则是：

- 太大：噪音多；
- 太小：上下文不够；
- 有重叠：通常比完全不重叠更稳。

7 核心技术四：向量化与本地向量数据库

7.1 什么是向量化

“向量化”就是把一段文本变成一个数字数组。项目里使用的是中文嵌入模型：

`shibing624/text2vec-base-chinese`

在报告中，它也被标注为 768 维向量模型。
你可以把向量理解成“文本在数学空间中的坐标”。相似的句子，坐标会更接近。

7.2 Embedding 模型在项目中怎么用

对应函数是：

`get_embedder()`

它采用了**懒加载**：

- 第一次用到时才加载模型；
- 避免程序一启动就占很多内存。

同时还有：

`free_embedder()`

用于释放内存。这说明作者已经考虑到本地机器资源有限的问题。

7.3 向量数据库为什么选 ChromaDB

项目选择的是本地持久化向量库 `ChromaDB`。好处是：

- 不需要单独部署服务器；
- 可以直接保存在本地目录 `index\_storage/`；
- 适合个人电脑、本地实验和课程项目。

集合初始化函数是：

`get_chroma_collection()`

这里还设置了：

`metadata=\{"hnsw:space": "cosine"\}`

意思是向量相似度主要按**余弦相似度**来理解：

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|\|\mathbf{d}\|}$$

其中 $\mathbf{q}$ 是查询向量，$\mathbf{d}$ 是文档向量。两个向量方向越接近，值越大，语义越相似。

7.4 索引建立的流程

`build_index()` 的逻辑可以概括成：

1. 如果索引已有内容，就直接复用；
2. 如果没有，就从 `sources/` 读取所有 `.txt`；
3. 每个文件分块；
4. 每个块做 embedding；
5. 把向量、文本、元数据一起写进 ChromaDB。

这里的元数据包括：

- `title`：文献标题；
- `file`：来源文件名。

这让后续报告可以告诉用户“这段文本来自哪一部书”。

8 核心技术五：混合检索与重排

8.1 为什么不是只做向量检索

如果只做向量检索，模型可能把“语义很像但人物年份不对”的段落也召回出来。
比如：

- 都讲“诸侯”“出兵”“周室危机”；
- 但不是同一位王、不是同一年、不是同一个人物。

对历史文本来说，这种错配非常致命。

8.2 项目做了哪三层检索

函数 `retrieve(query, top\_k=3)` 是全项目最关键的技术之一，它实际上做了三层事：

1. **实体与事件驱动的关键词召回**
2. **向量语义召回**
3. **局部片段级重排**

8.3 第一层：特征提取

项目会先从查询中抽出一组“检索锚点”。这一步由 `_extract_query_features(query)` 完成。

它抽取的对象包括：

- rulers：君主称号，如“顯王”；
- years：年份，如“三十三年”；
- names：人名，如“顏率”；
- states：国家名，如“秦”“周”“齊”；
- events：事件词，如“九鼎”“為諸侯”；
- phrases：较长短语；
- bigrams：二元字串集合，用于估计局部重合度。

这一步非常像“把一句话压缩成检索用的结构化线索”。

8.4 第二层：关键词加权打分

函数 `_score_keyword_candidate()` 不是简单数词频，而是给不同实体不同权重。

例如源码中，权重大致是：

- ruler：3.6
- year：3.0
- name：2.8
- phrase：1.8
- event：0.8

这说明项目认为：

> 在历史文本检索里，“是谁、哪一年”通常比“有没有泛泛的战争词”更重要。

8.5 第三层：语义召回

项目把查询转成向量后，会去 ChromaDB 里找最相近的若干候选块。代码中：

- `TOP\_K = 3`
- `SEMANTIC\_CANDIDATES = 12`

也就是说，它不是只取最终 3 条，而是先多拿一些候选，再做后续筛选。

8.6 第四层：局部片段抽取

一个向量块可能仍然偏长，所以项目还会调用：

```
_extract_best_snippet(doc_text, query, query_features)
```

它会在候选文档内部滑动窗口，找到最像查询的那一小段。
为什么这一步重要？因为用户真正要看的是：

> 最能说明问题的证据片段，而不是一大段长文。

8.7 第五层：窗口评分与惩罚机制

`_score_text_window()` 会综合多个因素打分：

- coverage：重要实体命中的覆盖率；
- ngram overlap：字串重合度；
- sequence ratio：序列相似度；
- bonus：关键实体命中后的加分；
- penalty：人物、年份、国别不一致时的惩罚。

源码里的公式近似可以写成：

$$\text{window_score} = 0.58 \cdot \text{coverage}0.24 \cdot \text{ngram}0.18 \cdot \text{sequencebonus} - \text{penalty}$$

最终重排分数近似为：

$$\text{final_score} = 0.28 \cdot \text{semantic}0.27 \cdot \text{keyword}0.45 \cdot \text{window} - 0.05 \cdot \text{penalty}$$

这说明项目的思想很明确：

> 真正决定名次的，不是“整段文档看起来像不像”，而是“文档里最关键的局部片段到底像不像”。

8.8 这一步教会我们什么

这一步是整份项目中最像“算法设计”的部分。它说明：

- 检索系统不一定追求最复杂模型；
- 只要特征设计合理、重排逻辑清楚，也能做出很强的效果；
- 面向具体任务，规则和向量往往不是对立关系，而是互补关系。

9 核心技术六：Diff 差异比对

9.1 为什么要做 Diff

检索找到来源后，用户接下来最关心的不是“像不像”，而是：

> 司马光到底改了什么？

所以项目没有停在检索，而是继续调用：

```
compute_diff(source_text, target_text, source_name)
```

9.2 技术原理是什么

这里使用的是 Python 标准库 `difflib`，本质上是比较两个序列之间的差异。

项目先把文本切成句子，再调用 `unified\_diff`。于是它得到的不是逐字乱比，而是更接近“句子层面的增删改”。

9.3 为什么输出成 HTML

因为前端用的是 Gradio，而 Gradio 的 Markdown/HTML 展示很方便。项目把差异包装成：

- 绿色：新增；
- 红色删除线：删减；
- 灰色：上下文。

这让一个本来偏算法的结果，变成了用户一眼就能读懂的可视化结果。

10 核心技术七：本地 LLM 解释层

10.1 为什么检索之后还要 LLM

检索和 Diff 能告诉你“发生了什么变化”，但不擅长直接回答：

- 这种变化可能意味着什么？
- 它反映了怎样的编纂取向？
- 它和北宋政治语境可能有什么关系？

这些问题需要**解释能力**，所以项目引入了本地大模型。

10.2 项目如何调用大模型

函数是：

```
call_llm(prompt, timeout=120)
```

它并没有依赖复杂框架，而是直接通过 HTTP 调用本地 Ollama 服务：

```
http://127.0.0.1:11434/api/generate
```

模型名配置为：

```
qwen2.5:3b
```

这说明项目走的是**轻量本地化路线**：不用云 API，不上传数据，适合个人电脑运行。

10.3 Prompt 设计有什么特点

项目不是一句“帮我分析一下”，而是给了明确结构：

1. 删减内容；
2. 新增/改写内容；
3. 语言风格变化；
4. 整体改写意图；
5. 一句话总结。

这说明一个重要事实：

> 大模型输出质量，不只取决于模型大小，也取决于你把任务拆得是否清楚。

10.4 为什么强调“本地”

本地 LLM 的价值在这个项目里至少有三点：

- 隐私更好，文本不出本机；
- 适合离线场景；
- 对课程展示来说，系统更完整，依赖更少。

11 核心技术八：Web 前端与交互封装

11.1 为什么还需要 Gradio

如果只有命令行，技术同学能用，但普通同学、老师、演示场景就不方便。

因此项目在 `app.py` 中用 Gradio 做了一个 Web UI。它提供了：

- 文本输入框；
- 示例输入；
- “开始分析”按钮；
- 分析报告展示区。

11.2 这不是“花哨界面”，而是产品化

这一层的意义是把算法变成可使用的工具。

从工程角度看，它完成了三件事：

1. 统一输入输出；
2. 让用户不必理解命令行；
3. 把复杂流程封装成“一次点击”。

11.3 一个值得注意的工程细节

`app.py` 里专门改写了部分 `httpx` 请求逻辑，对 `localhost` 和 `127.0.0.1` 禁用代理。

这说明作者遇到过真实环境问题：有些系统代理会误伤本地回环请求，导致 Gradio 启动失败或自检异常。

也就是说，**核心技术不只在算法里，也在这些“程序真正跑起来”的细节里。**

12 核心技术九：鲁棒性与工程质量

12.1 懒加载与资源控制

项目不是一上来就把所有模型都塞进内存，而是按需加载 embedding 模型，这对于普通笔记本非常重要。

12.2 持久化索引

索引一旦建好，不需要每次重建。这样启动速度更快，也更接近真实系统。

12.3 错误恢复

`_call_collection()` 和 `_collection_count()` 体现了一个工程细节：如果 Chroma collection 句柄失效，就自动重连一次。

这说明作者考虑的不是“代码理想情况下能跑”，而是“偶发故障时能不能恢复”。

12.4 测试

项目有一组测试文件 `tests/test_retrieval_helpers.py`。它重点测试了：

- 按语剥离；
- 繁简混合查询；
- 关键词打分是否偏向真正的实体；
- 局部片段抽取是否足够紧凑；
- 文本清洗是否能去掉 Wikisource 噪音；
- collection 丢失后的恢复。

这很值得初学者学习，因为测试不只是“保证没报错”，更是把关键设计意图固定下来。

13 从头到尾走一遍完整样例

13.1 样例输入

威烈王二十三年，初命晉大夫魏斯、趙籍、韓虔為諸侯。

13.2 系统内部会发生什么

1. 先检查输入够不够长；
2. 如果有“臣光曰”之类的按语，就先剥离；
3. 查看本地索引是否存在；
4. 抽取查询锚点，如“威烈王”“二十三年”“魏斯”“趙籍”“韓虔”“為諸侯”；
5. 先从所有史料块中做关键词召回；
6. 再做向量相似度召回；
7. 在候选文档内部截取最相关的小片段；
8. 输出 Top-K 结果；
9. 对每条结果做 Diff；
10. 把“原史料 + 通鉴文本”一起喂给本地 LLM；
11. 生成最终 Markdown 报告。

13.3 这一步最想让你看懂什么

不是“函数很多”，而是：

> 每个函数都只负责解决链条中的一个具体问题，最后再由主函数把它们串起来。

这就是软件工程中常说的**模块化**。

14 这套技术方案为什么成立

14.1 它抓住了任务本质

这个项目之所以合理，是因为它没有把“古籍分析”误当成一个纯聊天问题，而是识别出这个任务真正依赖三种能力：

1. 找到史料；
2. 对齐文本；
3. 解释差异。

14.2 它没有迷信大模型

项目里最重的“智能”并不全在 LLM 上。很多决定效果的关键步骤都是传统工程方法：

- 清洗；
- 分块；
- 特征抽取；
- 规则加权；
- 重排；
- Diff。

LLM 主要负责最后的高层解释，而不是包办一切。

14.3 它适合教学

这个项目特别适合作为新生入门案例，因为它把很多现代 AI 系统里的关键概念都串起来了：

- 文本处理；
- 信息检索；
- 向量表示；

- 数据库；
- Web 应用；
- Prompt 设计；
- 软件工程。

15 局限与改进方向

一份好的讲义不能只讲优点，也要讲边界。
目前这个项目的局限主要有：

1. 史料覆盖范围仍可能不完整；
2. 规则抽取对个别古文句式可能失效；
3. 向量模型是通用中文模型，不一定最懂先秦古文；
4. LLM 的历史解释仍然可能出现过度推断；
5. Diff 主要反映文本层面的改动，不等于严格的文献学考证。

可以继续优化的方向包括：

- 引入更完整的《左传》《国语》《通鉴考异》；
- 训练或替换更适合古汉语的 embedding 模型；
- 做人物、年代、地名的专门知识库；
- 加入时间线可视化；
- 让结果支持导出 PDF 或学术报告格式。

16 给大一新生的最终总结

如果你以前没有接触过 AI 系统，请记住下面五句话：

1. 这个项目不是“直接问大模型”，而是“先检索、再比对、后解释”。
2. RAG 的核心不只是向量数据库，更是数据清洗、分块和重排。
3. 向量相似度能帮你找“意思接近”的文本，但历史任务还需要实体约束。
4. Diff 把“模型觉得像”变成“人眼能看懂哪里改了”。
5. 一个真正可用的 AI 项目，算法、工程、界面和测试同样重要。

如果把这套系统再压缩成一行，可以写成：

古籍智能比对 Agent = 干净史料数据 + 古文友好的分块方式 + 混合检索与重排 + 文本差异可视化 + 本地大模型解释。

附：建议课堂讲法

如果你要拿这份讲义给大一学生上课，可以按下面顺序讲：

1. 先讲“这个系统想解决什么历史学问题”；
2. 再讲“为什么不能只用关键词搜索”；
3. 再讲“什么叫向量、什么叫 RAG”；
4. 然后走一遍项目主流程；
5. 最后拿一个真实例子现场演示。

这样学生不会先被术语淹没，而是先理解问题，再理解技术。